

p0946r0

Towards consistency between `<=>` and other comparison operators

Published Proposal, 10 February 2018

This version:

<http://wg21.link/p0946r0>

Author:

[Richard Smith](#) (Google)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

The specification of the `<=>` operator reexamined the rules underlying comparison operators, and disallowed some cases that have historically been allowed, but error-prone, for the other relational and equality operators. This paper considers options for realigning the rules for these operators, whether by deprecating or removing the corresponding cases for those other operators or by revisiting the rules for the `<=>` operator. This paper does not provide specific guidance for some of these questions, instead aiming to promote discussion and a search for a better answer.

Table of Contents

1	Differences
1.1	Sign safety
1.2	Enum safety
1.3	Array safety
1.4	Null safety
1.5	Function pointer safety

Index

Terms defined by this specification

References

Informative References

§ 1. Differences

The rules for the built-in `<=>` operator, introduced in [P0515R3], do not match those for the built-in `<`, `>`, `<=`, `>=`, `==`, and `!=` operators (herein referred to as *two-way operators*) in a number of ways:

- [§1.1 Sign safety](#) If one operand is of a signed integral type, but the type in which the comparison would be performed would be an unsigned type, a `<=>` comparison is rejected unless the signed operand is a constant expression that evaluates to a non-negative value; the other forms of comparison compare the converted value (for example, `-1 < 2u` evaluates to `false` but `-1 <=> 2u` is ill-formed).
- [§1.2 Enum safety](#) If either operand is of enumeration type, a `<=>` comparison requires the other operand to have the same type; other forms of comparison merely require the other operand to have arithmetic or enumeration type.
- [§1.3 Array safety](#) If both operands are of array type, a `<=>` comparison between them is ill-formed. The behavior of the two-way operators is unclear in the standard wording, but the widespread implementation consensus is that the array-to-pointer conversion is applied to the operands prior to checking their types, which results in such comparisons being permitted.
- [§1.4 Null safety](#) If one operand is a null pointer constant (a `0` literal or `nullptr`) and the other is a pointer, the relational `<`, `>`, `<=`, and `>=` comparisons are ill-formed. But `<=>` comparisons are permitted, returning a `std::strong_ordering` with an unspecified value.
- [§1.5 Function pointer safety](#) Relational comparisons between function pointers are permitted, with unspecified results. But `<=>` produces `std::strong_equality` for such comparisons, providing no ordering information.

In most cases, `<=>` provides superior rules, derived from experience with the existing operators. However, these differences will be the source of confusion and ire from C++ programmers indefinitely if we do nothing to resolve them. These differences will be analyzed in turn below.

For the sake of exposition, examples below use a simple wrapper type with a defaulted `operator<=>` (this provides a simplified model of types like `pair` and `tuple`):

```
template<typename T> struct wrapper { T t; };
template<typename T> wrapper(T) -> wrapper<T>;
template<typename T, typename U> auto operator<==>(wrapper<T>, wrapper<U>) =
```

Reasonable user expectation would be that two-way comparisons between `wrapper<T>` and `wrapper<U>` behave the same as the corresponding two-way comparisons between `T` and `U`, but as we will see, the above rule differences defy this expectation.

§ 1.1. Sign safety

EXAMPLE 1

```
static_assert(-1 > 1u);           // assert passes
static_assert((-1 <==> 1u) > 0); // ill-formed
```

The rule used by `<==>` in this situation avoids a long-standing class of bugs. However, the corresponding case for other comparison operators is likely in use in a significant amount of code; examples such as

```
void f(std::vector<int> v) {
    for (int n = 0; n < v.size(); n++)
```

abound, and implicitly rely on such signed/unsigned comparisons. We cannot realistically reject or even deprecate this code due to the volume of such comparisons in existing code.

Neither the behavior of the existing operators nor the behavior of `<==>` is ideal. Designing from a clean slate, one appealing option would be to specify that these comparisons "just work": that is, that they always give the mathematically-correct results, as if no conversion were performed on the operands:

```
static_assert(-1 < 1u);           // OK
static_assert((-1 <==> 1u) < 0); // OK
static_assert(-1 < UINT_MAX);    // OK
```

However, we cannot simply give the `<==>` operator the mathematical meaning without creating a major inconsistency between it and the other comparison operators. Such inconsistency would also leak into other operators, through automatic rewriting of those operators into uses of `<==>`. Consider:

```
static_assert(-1 < 1u);           // fails
static_assert(wrapper{-1} < wrapper{1u}); // passes!
```

We also cannot reasonably give the mathematical meaning to both the `<=>` operator and the two-way operators; existing mixed-signedness comparisons are too prevalent for their meaning to be changed at this stage.

Our most obvious remaining options are either to remove this narrowing restriction from `<=>` (allowing these comparisons to continue to produce values different from their mathematical meaning), or to retain the status quo: that `<=>` has validity rules different from those of two-way operators.

One non-obvious alternative exists: we could deprecate evaluations of two-way operators whose results are different from their mathematical results, and encourage vendors of static and dynamic analysis tools to diagnose such comparisons. It is unlikely that any deprecation period will suffice to allow us to change the meanings of these operators to their mathematical meaning, but we can at least send a signal that such operations are discouraged.

In the absence of a clearly best option, this paper makes no recommendation on this issue, but we hope to have motivated a search for a better answer than the status quo.

§ 1.2. Enum safety

EXAMPLE 2

```
enum E { a = 3 };
static_assert(a == 3);           // OK
static_assert((a <=> 3) == 0);    // ill-formed

enum F { b = 3 };
static_assert(a == b);           // OK
static_assert((a <=> b) == 0);    // ill-formed

static_assert(a >= 1.34);        // OK
static_assert((a <=> 1.34) >= 0); // ill-formed
```

The `<=>` operator provides some safety when comparing values of enumeration type. However, this safety comes at the cost of disallowing reasonable comparisons between values of an unscoped

enumeration and those of its underlying type. This leads to disallowing cases such as:

```
enum Bits { Foo = 1, Bar = 2, Baz = 4 };
bool containsFooBar(const std::set<Bits, std::less<>> &set) {
    // Note that Foo | Bar is of integral type, not type Bits
    return set.count(Foo | Bar);           // OK
}
bool containsFooBar(const std::set<wrapper<Bits>, std::less<>> &set) {
    return set.count(wrapper{Foo | Bar}); // ill-formed!
}
```

We can distinguish (at least) three cases, each of which is currently ill-formed when using `<=>` but valid under the two-way comparison operators:

- Comparison between a value of one enumeration type and a distinct enumeration type. Such cases are very likely due to bugs, and requiring an explicit cast (for any comparison operator) seems prudent.
- Comparison between a value of an enumeration type and a value of a non-integral (eg, floating-point) type. Likewise, this is probably undesirable, but it is possible that an enumeration type could define enumerators representing values for which comparison against floating-point types is reasonable and intended. (For example: `enum Brightness { Min = 0, Max = 255 }; double red = Brightness::Max * std::exp(-k * t); if (red > Brightness::Max) red = Brightness::Max;`)
- Comparison between a value of an enumeration type and a value of an integral type. Such comparisons are extremely common, and most such cases are probably intended and correct.

In the first case, the best option would likely be to deprecate or remove the support for such two-way comparisons. In the third case, the rules for the `<=>` operator seem to prevent a large class of useful programs without a commensurate benefit, and should be revised to permit such comparisons. The middle case seems debatable, but is likely sufficiently rare that requiring an explicit cast is not overly onerous.

Suggested approach: deprecate two-way comparisons between enumeration types and floating-point types / distinct enumeration types. Permit three-way comparison between unscoped enumeration types and integral types.

We should also consider whether we wish any such deprecation to occur only for comparison operators, or more generally for any case where the usual arithmetic conversions are applied between an operand of enumeration type and an operand of floating-point or distinct enumeration type (for instance, `RGB::Red | HSV::Blue` or `4.0 * Suffix::Giga`).

§ 1.3. Array safety

EXAMPLE 3

```
int a[3], b[4];
int *p;
void f() {
    a < b;           // OK (implementation consensus)
    a <=> b;          // ill-formed

    a < p;           // OK
    a <=> p;          // OK

    a < nullptr;     // ill-formed (by DR 583)
    a <=> nullptr;    // ill-formed
}
```

The validity of a two-way comparison between two array operands is unclear in the current standard text. Current implementations permit it; the chain of reasoning that appears to be used to justify this is:

- Two-way comparison operators require prvalues as their operands, despite this being nowhere stated in the standard text.
- Therefore, operands of array type are decayed to pointers.
- Such decay occurs *before* the two-way comparison operators perform checks on the types of their operands, so they never see an array operand, only a pointer.

This reasoning appears to be inspired by the rules of C, where the rules are more explicit and admit only the above interpretation.

However, the above reasoning breaks down for the `<=>` operator, because its description explicitly specifies when to apply the array-to-pointer decay, strongly implying that such conversion should not be applied beforehand.

The rule used by the `<=>` operator appears to be the more appropriate one in this case. Equality and relational comparisons between two array objects seem highly unlikely to be desirable, and create the false impression of comparing the array contents rather than the decayed addresses. As such, we propose deprecating two-way comparisons where both operands are of array type.

§ 1.4. Null safety

EXAMPLE 4

```
int *p;
void f() {
    p == nullptr;    // OK
    p < nullptr;     // ill-formed, meaningless
    p <=> nullptr;    // OK, std::strong_ordering,
                    // value unspecified if p non-null!

    nullptr == nullptr; // OK, true
    nullptr < nullptr;  // ill-formed
    nullptr <=> nullptr; // OK, std::strong_equality::equal
}
```

The resolution of core issue 583 made relational comparisons against null pointer constants ill-formed. Such constructs have always been ill-formed in C, and appear likely to have only ever been valid in C++ due to a wording oversight.

However, the `<=>` operator oddly produces a `std::strong_ordering` result when comparing a null pointer constant against an object pointer, producing `std::strong_ordering::equal` when the pointer is null and an unspecified value (which could even be `std::strong_ordering::equal`!) otherwise. This seems to also be merely a wording oversight.

Suggested approach: change the `<=>` operator to produce `std::strong_equality`, rather than `std::strong_ordering`, for comparisons between a null pointer constant and an object pointer.

§ 1.5. Function pointer safety

EXAMPLE 5

```
using Func = void();
Func *p;
Func *q;
Func &g;
void f() {
    p == q;    // OK
    p < q;     // OK?! value unspecified if p != q
    p <=> q;    // OK, std::strong_equality

    p == f;    // OK
    p < f;     // OK (implementation consensus)
    p <=> f;    // OK, std::strong_equality

    f == g;    // OK (implementation consensus)
    f < g;     // OK (implementation consensus)
    f <=> g;    // ill-formed
}
```

The implementation-consensus cases here are analogous to the array cases discussed above; the wording is not completely clear that these cases are valid. However, in the function case, permitting such comparisons seems less harmful: there is little risk of someone believing that the contents of a function rather than its address would be compared. There is a different risk, namely that the user may have intended `f() == g()` instead of `f == g`, but such heuristic checks are best left to quality-of-implementation compiler diagnostics.

The more pressing concern is that of relational comparisons between function pointers. There are two sensible possibilities here: either we should require a useful total ordering over such pointers, exposed by both the relational operators and by the `<=>` operator, or we should simply disallow such comparisons. We could consider making this choice implementation-defined, as we do for the choice to permit casting between function pointer and object pointer types. But we should not make different decisions for relational ordering operators and the `<=>` operator.

Suggested approach: deprecate relational comparisons between function pointers. Clarify that equality comparisons between two functions or references to function is valid.

It is worth noting that `std::less` produces a strict total order for function pointers, even though the `<` operator does not specify the result for any unequal comparison. Code using, say,

`std::set<Func*>` is guaranteed to work today, and it would be reasonable to expect it to continue working in the future. This may require adding a `std::less` specialization for function pointers if relational function pointer comparisons are ever removed, for implementations that implement `std::less` in terms of `<` today. (However, code using types compounded from function pointers, such as `std::set<std::tuple<Func*>>`, would still transition from being valid-but-unspecified to being ill-formed, unless we solve the more general issue that associative container ordering should permit more types than `<` does.)

§ Index

§ Terms defined by this specification

[two-way operators](#), in §1

§ References

§ Informative References

[P0515R3]

Herb Sutter, Jens Maurer, Walter E. Brown. [Consistent comparison](https://wg21.link/p0515r3). URL: <https://wg21.link/p0515r3>